

MovieMate

Exploring Conversational AI for Intelligent Movie Search and Recommendations

1. Project Overview

The goal of this project is to design and experiment with a conversational AI system that helps users explore movie information through natural language interactions. Instead of relying on traditional keyword-based search interfaces, users interact with an intelligent assistant capable of understanding questions, interpreting preferences, and retrieving relevant movie information.

This project investigates how modern Natural Language Processing (NLP) techniques and Large Language Models (LLMs) can be combined with structured movie datasets to create an intuitive movie discovery system. The system will not only retrieve information but also engage in conversational interactions such as recommendations, follow-up queries, and contextual refinements.

Rather than building a simple query engine, the emphasis of this project is exploratory: understanding how conversational AI systems can be designed to assist users in navigating large entertainment datasets while maintaining responsiveness, interpretability, and scalability.

2. Background and Motivation

The growth of digital entertainment platforms has led to a massive increase in available content. Users often struggle to efficiently discover movies that align with their interests, mood, or contextual preferences. While databases such as IMDb provide extensive movie information, they typically rely on structured browsing or keyword search.

Traditional search approaches impose several limitations:

- Users must know **exact keywords or movie titles**.
- Complex queries such as *"Suggest movies similar to Interstellar but less than 2 hours long"* are difficult to express.
- Discovery often requires navigating multiple pages and filters.

Conversational AI presents an alternative paradigm where users can interact naturally with systems. By combining natural language understanding with retrieval-based search mechanisms, it becomes possible to create systems that behave more like knowledgeable assistants rather than static search tools.

This project explores the design of such a system using a combination of structured movie data and modern language models.

3. Problem Statement

Movie discovery platforms typically rely on static search interfaces and pre-computed recommendation systems. While these approaches work well for large-scale streaming services, they often lack interactive capabilities.

Users frequently want to ask questions such as:

- "Suggest sci-fi movies released after 2010"
- "Movies similar to Inception"
- "Good drama movies with high IMDb ratings"
- "What movies did Christopher Nolan direct?"

Traditional interfaces cannot easily handle such flexible natural language queries.

The problem addressed in this project is therefore:

How can a conversational AI system be designed to interpret natural language movie queries and retrieve relevant information from a structured movie dataset in an efficient and interactive way?

The project focuses on exploring architectures that combine language models with vector-based retrieval methods.

4. Project Objectives

The project is structured around several exploratory objectives.

4.1 Natural Language Movie Search

Enable users to query the system using conversational language rather than structured filters.

Examples:

- "Show me highly rated thriller movies"
- "Recommend some feel-good movies"
- "Movies starring Leonardo DiCaprio"

The system should interpret these queries and retrieve relevant movies from the dataset.

4.2 Conversational Interaction

The chatbot should support **multi-turn conversations**, allowing users to refine their queries.

Example interaction:

User: Suggest action movies.

Bot: Here are some popular action movies...

User: Only those released after 2015.

Bot: Updated recommendations...

Handling conversational context is an important aspect of modern chatbot systems.

4.3 Personalized Exploration

If user interactions are stored, the system can potentially adapt recommendations based on past queries or interests.

Examples:

- Tracking previously searched genres
- Recommending similar movies
- Identifying user preference patterns

This component may be explored conceptually even if a full recommendation system is not implemented.

4.4 Intelligent Information Retrieval

The system should retrieve movie details such as:

- Ratings
- Genre
- Director
- Cast
- Duration
- Year of release

These attributes will be used to construct informative responses to user queries.

5. Machine Learning and NLP Formulation

From a machine learning perspective, this project combines two key components:

1. Natural Language Understanding

User queries must be interpreted by an NLP system capable of extracting intent and semantic meaning.

For example:

"Best comedy movies after 2010"

This query contains multiple semantic constraints:

- Genre = Comedy
- Year > 2010
- High rating preference

Large Language Models can help interpret such queries and transform them into structured search operations.

2. Retrieval-Based Search

Once the intent is interpreted, the system must retrieve relevant movies from the dataset.

Vector embeddings can be used to represent movie descriptions and user queries in a semantic space. Similarity search can then identify the most relevant matches.

This architecture is commonly referred to as **Retrieval-Augmented Generation (RAG)**.

6. Technical Approach

The system will be developed through several stages.

6.1 Data Collection and Preprocessing

The project uses a movie dataset derived from IMDb.

Initial steps include:

- Loading the dataset into a Python environment
- Inspecting the dataset structure
- Handling missing values
- Cleaning textual attributes such as genre, cast, and titles

Basic preprocessing may involve:

- Normalizing text fields
 - Removing inconsistencies
 - Preparing textual data for embedding generation
-

6.2 Exploratory Data Analysis (EDA)

EDA will help understand the characteristics of the dataset.

Possible analyses include:

- Distribution of movie ratings
- Frequency of genres
- Relationship between release year and ratings
- Common actors and directors

Visualization tools such as **Matplotlib** or **Seaborn** may be used to generate insights.

6.3 Feature Representation

Textual attributes such as movie titles, genres, and summaries can be converted into vector embeddings.

Embeddings allow semantic similarity comparisons between:

- User queries
- Movie metadata

These vectors form the basis for efficient similarity search.

6.4 Similarity Search with Vector Databases

To retrieve relevant movies efficiently, vector similarity search techniques will be explored.

One commonly used approach is **FAISS (Facebook AI Similarity Search)**, which allows fast nearest-neighbor search in high-dimensional vector spaces.

Workflow:

1. Convert movie metadata into embeddings.
 2. Store embeddings in a vector index.
 3. Convert user queries into embeddings.
 4. Retrieve the most similar movie vectors.
-

6.5 Response Generation with Large Language Models

Once relevant movies are retrieved, a language model can be used to generate natural conversational responses.

For example:

User Query:

"Suggest some good adventure movies"

Retrieved Movies:

- Movie A
- Movie B
- Movie C

The LLM converts this information into a human-readable response such as:

"Here are some highly rated adventure movies you might enjoy..."

6.6 Conversational Interface

A lightweight interface will allow users to interact with the chatbot.

Possible frameworks include:

- **Gradio**
- **Streamlit**

These tools enable quick development of web-based interfaces for machine learning applications.

7. Data Acquisition and Dataset Construction

Unlike projects that rely on a pre-curated dataset, this project encourages students to **build their own dataset** by exploring publicly available movie data sources. The goal is to expose the practical challenges of collecting, cleaning, and structuring real-world data.

You are encouraged to explore multiple approaches for obtaining movie data, including:

7.1 Web Scraping

Movie metadata can be collected by scraping publicly available webpages that list movie information. For example, websites that provide movie ratings, cast information, or genre classifications can serve as potential sources.

Typical steps include:

- Identifying relevant webpages containing movie information
- Extracting structured fields such as title, rating, year, genre, cast, and director
- Parsing HTML content using Python libraries such as **BeautifulSoup** or **Scrapy**
- Storing the extracted data in structured formats such as CSV or JSON

Ensure that scraping is performed responsibly and in accordance with website usage policies.

7.2 Using Public APIs

Several movie platforms provide APIs that allow developers to programmatically retrieve movie data. These APIs typically provide structured access to movie metadata such as:

- Movie titles
- Ratings
- Plot summaries
- Cast and crew information
- Genre classification
- Posters and media links

Examples of sources include:

- Movie database APIs
- Open movie datasets
- Public entertainment data repositories

Using APIs often simplifies data collection since the information is already structured.

7.3 Exploring Existing Open Datasets

Explore existing open datasets related to movies and entertainment. These datasets can serve as a starting point or can be combined with scraped data to enrich the final dataset.

Possible sources include:

- Open movie datasets
- Kaggle datasets related to movies
- Public media metadata repositories

7.4 Dataset Construction

Regardless of the source, the final dataset should contain a structured table with attributes such as:

- Title – Name of the movie
- Rating – Average audience or critic rating

- Year – Release year
 - Genre – Movie categories
 - Director – Director of the movie
 - Cast – Key actors
 - Duration – Runtime of the movie
 - Additional metadata such as summaries or posters if available
-

8. System Architecture (Conceptual)

The system architecture can be conceptualized as a pipeline consisting of several components:

User Query



Natural Language Understanding (LLM)



Query Embedding Generation



Vector Similarity Search



Movie Retrieval



Response Generation



Chat Interface

This modular architecture allows different components to be improved independently.

9. Project Deliverables

The final submission will include a Jupyter Notebook containing the following components.

1. Dataset Exploration

- Description of dataset features
- Summary statistics
- Observations from the dataset

2. Exploratory Data Analysis

- Visualizations of ratings and genres
- Identification of common movie trends

3. Data Preprocessing

- Cleaning and preparing the dataset
- Handling missing values

4. Embedding and Retrieval

- Generating vector representations
- Implementing similarity search

5. Conversational Movie Chatbot

- Integrating retrieval with an LLM
- Generating natural language responses

6. Interactive Interface

- Simple web interface for user interaction

7. Evaluation and Reflection

- Observations about chatbot performance
- Limitations of the current approach

10. Expected Outcomes

At the end of this project, the following outcomes are expected:

- A working conversational prototype for movie exploration
- A system capable of answering natural language movie queries
- An understanding of how retrieval and language models can be integrated

Beyond the implementation, the project also aims to provide insights into:

- Designing conversational information systems
 - Integrating structured data with language models
 - Practical challenges in building AI-powered assistants
-

11. Possible Extensions

Several extensions may be explored in future iterations:

- Incorporating **movie plot summaries** for deeper semantic retrieval
 - Adding **multimodal search** using posters or trailers
 - Implementing **personalized recommendation models**
 - Exploring **agent-based architectures** for more complex interactions
-